Scaling Laws for Transformer Language Models on SVG Code

CS-GY 6923 Extra Credit Project

Taaha Bin Mohsin

05/01/2026

Abstract

This report explores scaling laws for decoder-only transformer language models trained on Scalable Vector Graphics (SVG) code. I cleaned and tokenized 1.17M SVGs (about 130M BPE tokens) from the StarVector datasets, trained five models ranging from roughly 800K to 85M non-embedding parameters, and fit a power-law scaling curve $L = a \cdot N^{-\alpha} + c$. Under standard parameterization (SP), the fitted exponent $\alpha = 0.126$ with $R^2 = 0.984$, steeper than the $\alpha \approx 0.076$ Kaplan et al. reported for natural language. I compared SP against Maximal Update Parameterization (μP) on both the original model family and a width-only family. μP passed the coordinate check and matched SP's LR optimum at base width, but underperformed SP at wider scales due to its effective LR reduction under a tight step budget. For generation, I retrained XL on a more diverse corpus (155M tokens, 12% multi-element SVGs from the non-simplified `svg-stack` dataset) for 4 epochs, getting val loss 1.86 (perplexity 6.47), 8.3% unconditional and 33.3% prefix-conditioned render rate.

1 Introduction

In this project, I look at SVG code as a test domain. SVG is XML-based with strict syntax, so invalid output can be detected automatically. It has hierarchical structure more like code than prose, and outputs render to images, so you can evaluate quality both quantitatively and visually. The vocabulary is also smaller and more constrained than English text.

I also investigate hyperparameter transfer across model scales. Under standard parameterization (SP), the best learning rate for a small model is usually not optimal for a larger one. μP from Yang et al. (2022) reparameterizes the network so that the optimal learning rate stays the same as you widen the model. I test whether this actually helps on SVG data.

The main contributions are: (1) a complete data-preprocessing pipeline for SVG code including normalization, deduplication, validation, and BPE tokenization; (2) a scaling study of decoder-only transformers from ≈1M to ≈88M parameters with a fitted power-law exponent; (3) a comparison of SP against μP with learning-rate sweeps under each; (4) an extrapolation to predict loss at 10× scale; and (5) quantitative and qualitative evaluation of generated SVGs.

I used Anthropic's Claude (Opus and Sonnet) to help write code and debug issues as well as help with phrasing and refining the report, but wrote the report myself, which the instructor verbally authorized during lecture. The design decisions, experimental choices, and analysis are my own. I

ran all code on Google Colab Pro. All major decisions are logged in decision_log.md, which I wrote over the course of two weeks while doing this.

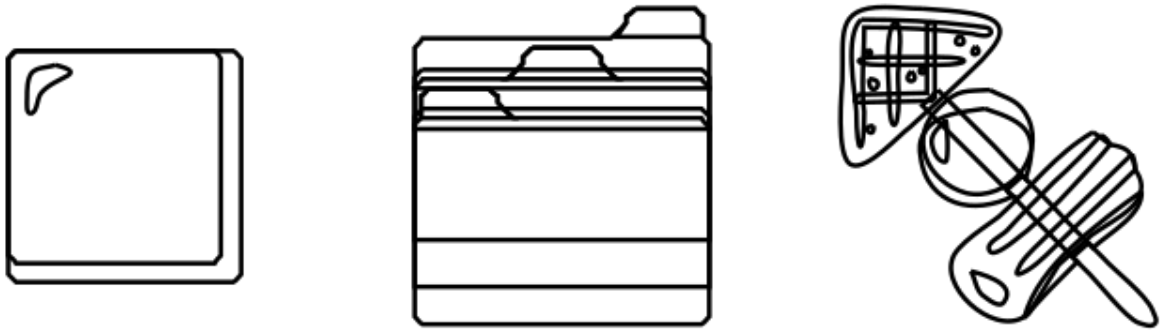
2 Data

2.1 Datasets

The primary dataset is `starvector/svg-icons-simple` (Rodriguez et al., 2023), about 89,370 simplified SVG icons. The icons alone were not enough to hit the 100M token target, so I added `starvector/svg-emoji-simple` (4,114 emoji SVGs) and a 65% subsample of `starvector/svg-fonts-simple` (1,134,108 font glyph SVGs). This gave me 1,218,656 raw SVGs and roughly 130M tokens after cleaning. The fonts are about 93% of the data by file count.

I used this corpus for the scaling study (Parts 2-3) and for an initial generation run (Part 4). The generation results were decent but somewhat limited: the model only ever produced single-path font glyphs, because all three StarVector "simplified" datasets normalize every SVG to a single `<path>` element. To see if a more diverse training mix would help, I built a second corpus by also pulling from the non-simplified `starvector/svg-stack`, which preserves multi-element SVGs with `<circle>`, `<rect>`, `<g>`, etc. I streamed 100K such SVGs (filtered to those with at least one non-path element, under 5000 characters) and combined them with the base data. Section 2.5 describes the composition of this diversified corpus. The report presents generation results from both runs side by side.

Figure 1 shows three SVGs from the cleaned dataset at different complexity levels.



(a) Simple (936 chars) (b) Medium (2,404 chars) (c) Complex (5,232 chars)

Figure 1: Dataset SVGs at three complexity levels. All are single `<path>` elements since the StarVector simplified datasets normalize every SVG to a single path.

2.2 Cleaning and Normalization

Each SVG is processed in order: strip comments and metadata, extract the `<svg>` root, round numeric values to one decimal place, collapse whitespace, validate XML with `lxml`, discard SVGs under 50 characters, and deduplicate by MD5 hash. Rounding coordinates to 1 decimal place reduced a representative icon from 2,235 to 919 characters (59% compression) while remaining sub-pixel at 24×24 viewBox.

The pipeline processed 1,218,656 SVGs. Deduplication removed 48,031 files, and after tokenization

another 87 were filtered for exceeding the 1,024-token maximum, leaving 1,170,538 (96.1%). No files were rejected for invalid XML or for being under 50 characters, which is expected since the StarVector team already ran their own cleaning pass on these datasets. A render check using CairoSVG on the cleaned data confirmed a 100% render success rate.

2.3 Tokenization

I trained a BPE tokenizer using the HuggingFace `tokenizers` library with a byte-level pre-tokenizer and vocabulary size of 4,096. At 130M training tokens, each token sees ∼32K occurrences on average. I chose 4,096 as a balance: 1K would make sequences very long, while 8K+ would have too many rare tokens for good embeddings. Four special tokens are used: `<PAD>` (0), `<BOS>` (1), `<EOS>` (2), `<UNK>` (3).

One interesting thing the tokenizer learned: the entire SVG header and common path preamble got merged into a single token (ID 1024), compressing a typical 919-character icon down to about 96 tokens. The tokenizer was trained on a random 150K-SVG sample because Colab's memory limits killed the BPE merge computation on the full 1.17M corpus.

2.4 Splits and Final Statistics

I split by file (not by token position) to prevent data leakage.

Table 1: Dataset splits and statistics.

Split	# Files	# Tokens
Train	1,147,128	130,404,653
Val	11,705	1,326,073
Test	11,705	1,341,361
Total	**1,170,538**	**133,072,087**

Sequence lengths have a median of 98 tokens (P95: 246, P99: 447). Only 65 SVGs exceed the 1,024-token context window and are filtered out. Figure 2 shows the distribution.

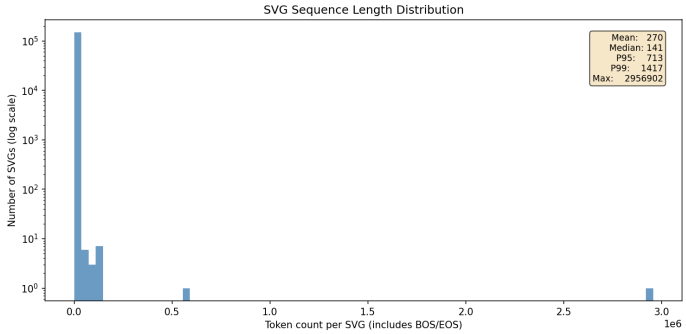


Figure 2: Sequence length distribution (tokens). Most SVGs are under 200 tokens; P99 is 1417.

2.5 Diversified Corpus for Generation

The original corpus is 93% font glyphs. All StarVector "simplified" datasets normalize every SVG to a single `<path>` element, so the model never sees multi-element structures like `<circle>`, `<rect>`,

or `<g>`. To get more diversity for generation, I built a second corpus by adding 100K multi-element SVGs streamed from the non-simplified `starvector/svg-stack` (filtered to SVGs containing at least one non-path element, under 5000 characters). After cleaning, 98,832 survived. I retrained the tokenizer on the combined corpus; it now learns `circle`, `rect`, `<g`, and `fill="#` as dedicated tokens. The diversified corpus has 155M training tokens (42% fonts, 36% stack-simple, 12% stack-multi, 10% icons). The scaling study (Parts 2-3) uses the original 130M-token corpus; only the generation model uses the diversified corpus alongside the original corpus for comparison.

3 Methods

3.1 Model Architecture

The model is a standard decoder-only Transformer following the GPT-2/nanoGPT pattern: pre-LayerNorm, causal multi-head self-attention, a two-layer FFN with GELU, learned positional embeddings, and weight tying between the token embedding and output projection. Context window is 1024 tokens. I borrowed the transformer block structure and weight-tying approach from nanoGPT (Karpathy). I wrote everything else in a custom implementation for the project.

Table 2: Model configurations.

Model	d_{model}	n_{layers}	n_{heads}	d_{ff}	Non-emb params
Tiny	128	4	4	512	793,344
Small	192	6	6	768	2,669,568
Medium	384	6	6	1536	10,647,552
Large	512	10	8	2048	31,524,864
XL	768	12	12	3072	85,056,000

3.2 Training Setup

All models train for exactly one epoch (≈130M tokens, 1,989 steps). Optimizer: AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1 (weight matrices only). LR schedule: 200-step linear warmup then cosine decay to 0.1× peak LR. Batch size: 64 sequences × 1024 tokens = 65,536 tokens/step. Mixed precision: bf16 on A100 (Google Colab Pro, 40 GB).

3.3 Learning Rate Sweep

I swept LR on Tiny only and transferred the optimum to all models without retuning. SP sweep: 7 LRs from 3e-5 to 3e-2 on a log scale. μP sweep: 9 LRs (same grid plus 1e-1, 3e-1). Each run trains for 2,000 steps. Selection criterion: lowest final validation loss.

3.4 μP Implementation

I implemented μP using the `microsoft/mup` package (`src/model_mup.py`). The main changes from the standard model: attention scaling uses scale = $\sqrt{\texttt{BASE_HEAD_DIM}}/d_{\text{head}}$ (matching SP at base width, giving the μP-prescribed $1/d$ scaling at wider widths). Also, the output head uses `MuSharedReadout` with weight tying, hidden weights are initialized with `mup.init.normal_` after `set_base_shapes` attaches `infshape` annotations, and the optimizer is `MuAdamW`. One non-obvious detail: the cosine LR schedule had to be applied as a multiplicative factor to each parameter group's base LR, not by overwriting all groups with a single scalar. I validated the implementation with the

coordinate check from the μP README, confirming that activation norms overlap across widths under μP and fan out under SP.

3.5 Best Model Training

After the 1-epoch scaling study, XL (val=3.1074) was clearly undertrained. Before building the diversified corpus, I first tried just training longer on the original font-heavy data: I extended XL to 4 epochs (5,967 steps) using `scripts/13_extend_xl.py`, saving to a separate checkpoint to preserve the 1-epoch weights for scaling comparisons. Val loss dropped from 3.10 to 2.24, but generation got worse in the ways that mattered. The unconditional samples (Appendix C) are actually cleaner than the 1-epoch versions as you can make out recognizable letterforms like a 7, V, W, D, and O, but this just means the model got better at fonts specifically. Prefix render rate dropped from 20% to 6.7%: only 1 out of 15 prefix completions rendered at all, and that one (Prefix 1 at $t = 1.0$) is a messy dark blob. The model doubled down on single-path font patterns and got even worse at multi-element structure. This is what told me the bottleneck was also the training data, not the amount of training, and motivated building the diversified corpus.

I then retrained XL from scratch on the diversified corpus for 4 epochs (9,484 steps, ≈621M tokens) with the same hyperparameters. Result: val loss 1.86, perplexity 6.47.

3.6 Evaluation

It was evaluated with three axes: test-set perplexity, SVG structural validity (XML parse by `lxml`), and SVG render rate (`cairosvg.svg2png` success). The render rate is the strongest test since cairosvg enforces attribute validity and path syntax beyond what XML parsing catches.

4 Results

4.1 Learning Rate Sweep

The SP sweep on Tiny has the expected U-shape: undertrained at lr=3e-5 (val≈6.2), minimum at lr=3e-4, and divergent at lr≥1e-2. Selected LR: 3e-4 (val=4.1948). The μP sweep finds the same optimum: lr=3e-4, val=4.2069, a 0.012-nat gap within run-to-run noise. This base-width parity confirmed the μP implementation is correct.

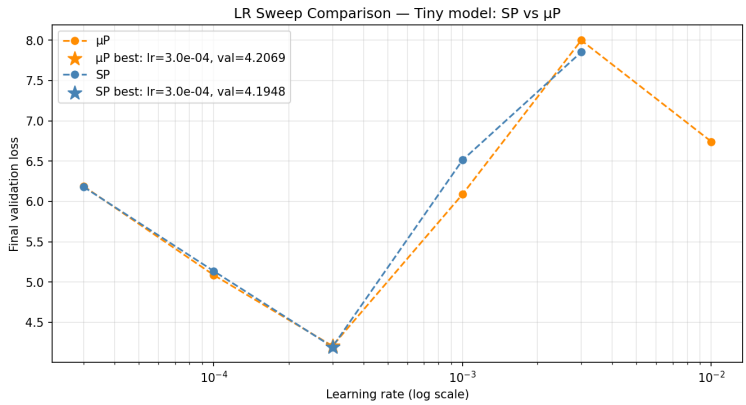


Figure 3: LR sweep comparison (SP vs. μP) on Tiny. Both find the same optimum at lr=3e-4.

4.2 Scaling Plot (Standard Parameterization)

Table 3: SP validation losses after 1 epoch, and training statistics (A100 40 GB, bf16).

Model	Non-emb params	Val loss	Wall time	Peak GPU	Tok/sec
Tiny	793,344	4.3178	1.5 min	7.3 GB	≈1,448K
Small	2,669,568	4.0401	3.4 min	14.2 GB	≈639K
Medium	10,647,552	3.4757	4.6 min	17.1 GB	≈472K
Large	31,524,864	3.2885	10.3 min	36.0 GB	≈211K
XL	85,056,000	3.1043	24.9 min	32.9 GB*	≈87K

*XL uses `-grad_accum 2` (32 seqs/micro-batch), halving peak memory.

Fitting $L = a \cdot N^{-\alpha} + c$ via nonlinear least squares gives: $L = 15.79 \cdot N^{-0.126} + 1.50, \quad R^2 = 0.984$. The fitted exponent $\alpha = 0.126$ is steeper than what Kaplan et al. found for natural language ($\alpha \approx 0.076$). I discuss reasons for this in Section 5.1.

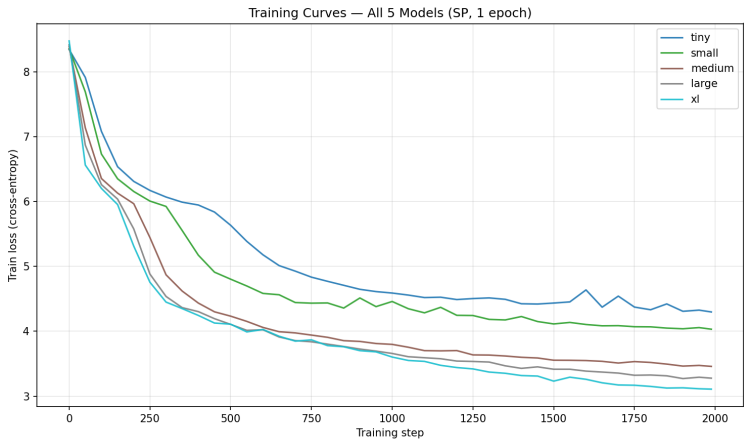


Figure 4: Training loss curves for all five SP models (1 epoch). Larger models achieve lower final loss.

4.3 SP vs. μP Scaling Comparison

Transferring lr=3e-4 from Tiny to all five model sizes under each parameterization:

Table 4: SP vs. μP final validation loss.

Model	Params	SP final	μP final	μP−SP
Tiny	793,344	4.3178	4.2668	−0.051
Small	2,669,568	4.0401	4.0661	+0.026
Medium	10,647,552	3.4757	4.0217	+0.546
Large	31,524,864	3.2885	3.6837	+0.395
XL	85,056,000	3.1043	3.5847	+0.480

Two power-law fits:

$$\text{SP:} \quad L = 15.79 \cdot N^{-0.126} + 1.50, \quad R^2 = 0.984$$

$$\mu\text{P:} \quad L = 7.08 \cdot N^{-0.037} + 0.00, \quad R^2 = 0.933$$

μP and SP coincide at base width (Tiny: 4.27 vs. 4.32, essentially tied). SP outperforms μP at Medium, Large, and XL by 0.4–0.55 nats, and scales more steeply ($\alpha = 0.126$ vs. 0.037).

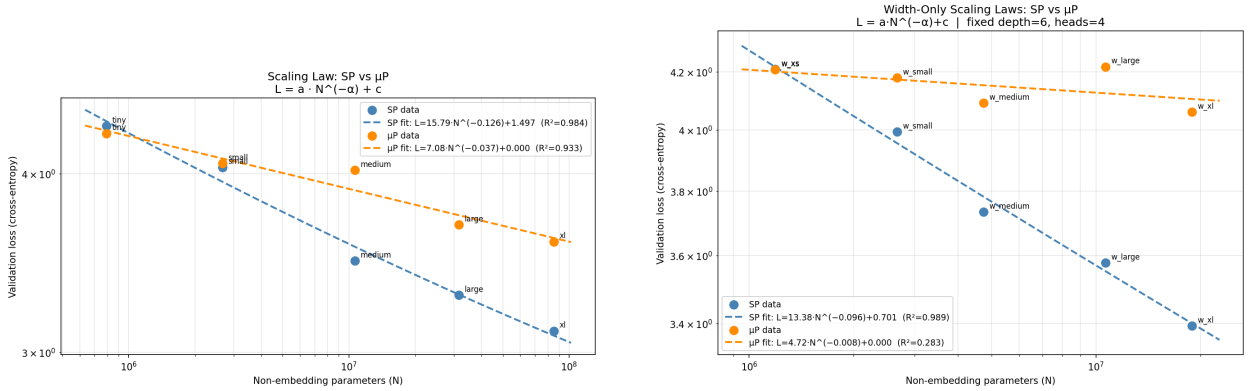
4.3.1 Width-Only Family (Supplementary)

To test whether the depth/heads confound explains μP's underperformance, I ran a second comparison on a width-only family: five models with fixed depth=6, heads=4, and only d_{model} varying (128 to 512). This is the clean setup μP is designed for, with unambiguous `width_mult` from 1 to 4.

Table 5: Width-only family: SP vs μP (1 epoch).

Model	d_{model}	Params	SP val	μP val
w_xs	128	1,189,888	4.2485	4.2586
w_small	192	2,669,568	3.9940	3.9814
w_medium	256	4,739,072	3.7336	4.0918
w_large	384	10,647,552	3.5782	4.2152
w_xl	512	18,915,328	3.3960	4.0590

SP scales cleanly ($\alpha = 0.096$, $R^2 = 0.989$), but μP remains nearly flat ($\alpha = 0.008$, $R^2 = 0.283$). So the depth confound is not the main issue here. It took me a bit of effort to figure out what was actually going on, but it comes down to how `MuAdamW` handles learning rates. It divides hidden-layer LRs by `width_mult`. That means w_xl, which has `width_mult`=4, is training its main parameters at an effective LR of 3e-4/4 = 7.5e-5. To see why that matters, I looked back at the SP sweep results: even lr=1e-4 gave val=4.98, which is way above the optimum of 4.25 at lr=3e-4. So 7.5e-5 is deep in undertrained territory. With only ∼2000 steps and cosine decay on top of that, the wider μP models just do not get enough updates to converge.



(a) Original family (varies depth, width, heads).

(b) Width-only family (fixed depth=6, heads=4).

Figure 5: Scaling law comparison: SP (blue) vs. μP (orange). SP scales more steeply in both families; μP underperforms at all wider sizes due to effective LR reduction.

4.4 Extrapolation to 10× Scale

Using the SP fit to predict loss at 851M non-embedding parameters (10× XL):

$$L(851\text{M}) = 2.68, \quad 95\% \text{ CI } [2.15,\ 3.21]$$

The confidence interval comes from the parameter covariance matrix that `scipy.optimize.curve_fit` returns. I would not put too much weight on this prediction though. Within about 3× XL it is probably in the right ballpark, but beyond that there are some reasons to be skeptical. For one, all five models only trained for a single epoch so the power law is really capturing "one-epoch loss" rather than converged loss. The model family also changes depth alongside width, which means a 10× model would need a new architecture choice that might not follow the same curve. On top of that, the learning rate I picked on Tiny may not be optimal at 851M parameters. And finally, the dataset is only 130M tokens, so at some point the model would just memorize it and loss would flatten out regardless of how many parameters one adds.

4.5 Generated Samples

I generated 30 samples from the diversified XL model: 15 unconditional (5 per temperature from $\{0.5, 0.8, 1.0\}$) and 15 prefix-conditioned (5 prefixes × 3 temperatures). I used the settings top-k=50, top-p=0.95, and max 1024 new tokens.

Table 6: XL model training results.

Model	Corpus	Steps	Val Loss	Perplexity
Original XL (1-epoch)	Font-heavy 130M tok	1,989	3.1074	22.31
Extended XL (4 epochs)	Font-heavy 130M tok	5,967	2.2427	9.39
Balanced XL (4 epochs)	Diverse 155M tok	9,484	**1.8630**	**6.47**

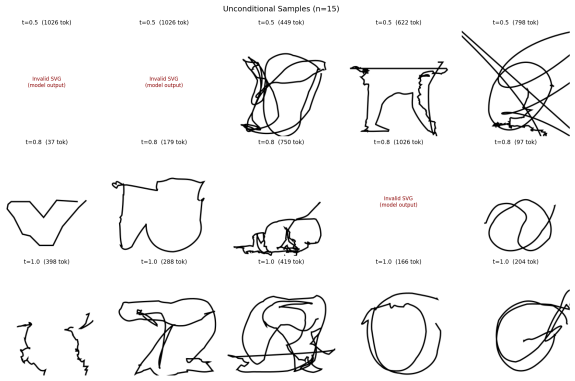
Table 7: Quantitative evaluation of generated samples.

Metric	Orig. uncond	Orig. prefix	Balanced uncond	Balanced prefix
XML valid	80.0%	20.0%	8.3%	33.3%
SVG renderable	80.0%	20.0%	8.3%	33.3%
Test perplexity	22.31 (orig. 1-ep)		6.47 (balanced 4-ep)	

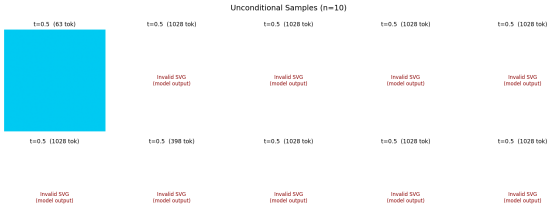
The diversified model achieves much lower perplexity (6.47 vs 22.31) and better prefix completion (33.3% vs 20.0%), but lower unconditional render rate (8.3% vs 80.0%). The lower unconditional rate is not worse modeling: the original model's high render rate mostly reflects a simpler single-path glyph prior from the font-heavy corpus. The diversified model attempts longer, multi-element SVGs, so most failures are truncation (hitting the 1024-token cap before closing tags) rather than malformed syntax.

The samples that do render from the diversified model look quite different from the original (Figure 6(b)): colored shapes and grouped rectangles, versus the original model's monochrome single-path strokes. For prefix completion at $t = 0.8$ (Figure 6(c)), the diversified model renders Prefix 1 (open path) as a house-like shape, while the other prefixes fail. At other temperatures the picture is more interesting: Figure 9 in the appendix shows Prefix 1 at $t = 0.5$ producing a house with window-like cutouts, Prefix 2 at $t = 1.0$ producing a red rectangle, and Prefix 3 at $t = 1.0$ producing an arrow.

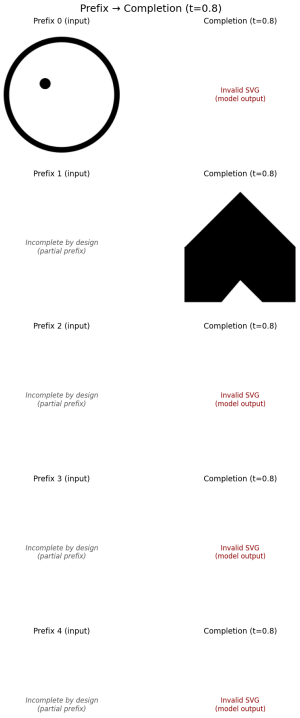
For the prefix-conditioned samples, Prefix 1 produces house-like shapes, Prefix 2 produces a colored rectangle, and Prefix 3 produces an arrow shape. The original model was trained on the font-heavy corpus, where 93% of examples were single-path glyphs, so it did not reliably render these multi-element SVG prompts.



(a) Original XL: mostly single-path glyphs. Render rate: 80%.



(b) Diversified XL: more varied outputs, but many hit the token cap. Render rate: 8.3%.



(c) Prefix completions from the diversified XL.

Figure 6: Qualitative samples from the original and diversified XL models. The diversified model produces more varied unconditional samples and meaningful prefix-conditioned completions, but its render rate is lower because many generations hit the 1024-token cap.

5 Discussion

5.1 Comparison to Natural Language Scaling

The SP exponent $\alpha = 0.126$ is steeper than the $\alpha \approx 0.076$ that Kaplan et al. (2020) reported for Transformers on natural language. I think there are two plausible reasons for this. First, SVG is a much smaller and more regular vocabulary than English, since it is just path commands, XML tags, and rounded coordinates, so extra parameters probably help more directly over the range I tested. Second, the exponent comes from single-epoch training, not converged models. Hoffmann et al. (2022) showed that scaling conclusions can change a lot when you vary the amount of training rather than holding it fixed, so $\alpha = 0.126$ is really specific to this single-epoch setup. Training longer might bring it down, but we would need a separate set of runs at different epoch counts to say for sure.

5.2 Why μP Underperformed

Under SP, the optimal learning rate shifts with model width because wider layers sum more terms per output, changing how large gradient updates are relative to the weights. μP fixes this by adjusting initialization, attention scaling, output projection, and per-layer learning rates so that effective update size stays constant regardless of width.

In my experiments, μP transfer works mechanically: no runs diverged, loss curves look clean, and at base width the SP and μP sweeps find the exact same optimal LR. But μP gives worse loss at every wider model size. I initially thought this was because the model family changes depth and head count alongside width. The width-only experiment ruled this out. Even with pure width scaling, SP gives $\alpha = 0.096$ while μP stays relatively flat ($\alpha = 0.008$).

I believe the actual cause is that `MuAdamW` divides hidden-layer LRs by `width_mult`. At w_xl (`width_mult`=4), the effective LR is 7.5e-5 for 93% of the parameters. The sweep showed that lr=1e-4 already gives val=4.98 (well above the optimal 4.25), so 7.5e-5 puts wider μP models in the undertrained zone. μP's theory assumes you train long enough for the lower LR to find the width-invariant optimum. The single-epoch regime here does not satisfy that assumption.

5.3 What the Model Learned

Generated SVGs consistently use correct viewBox, fill/stroke styling, and coordinate precision, showing the model learned SVG conventions pretty well. With the diversified corpus, some samples start with `<path>` (matching the still-dominant single-path pattern) while others generate multi-element structures with `<circle>`, `<rect>`, and `<g>` groups. Centering is partially learned (coordinates cluster in the 8-16 range within the 24×24 viewBox), but I did not see clear symmetry. Lower temperatures ($t = 0.5$, $t = 0.8$) produce cleaner shapes, but $t = 1.0$ produces the most varied and colorful outputs - the red rectangle and arrow in Figure 8 only show up at $t = 1.0$.

The original generation results made it clear that the bottleneck was not just model capacity, but also training data composition. Even the single-path font glyphs looked rough after just 1 epoch, so for a more complex training mix I trained for 4 epochs to give the model enough time to learn the harder multi-element patterns. Adding 12% multi-element SVGs and retraining the tokenizer was enough to unlock these completions, which suggests that for structured domains like SVG, curating the right training mix matters at least as much as scaling up the model. There is a tradeoff between coherence and diversity. For prefix completion, the model succeeds on tasks matching patterns in the training data (closing an open path, completing a group). Multi-element prefixes like a circle

plus an eye remain challenging because they require learning spatial relationships between elements, which needs more data and training than coordinate sequences within a single path.

5.4 Limitations and Challenges

Even with rebalancing, multi-element SVGs are only 12% of the corpus. The larger models are also still undertrained after 4 epochs, so the scaling exponent is probably inflated since they have more room to improve with additional training. μP's LR reduction makes this worse for wider models. A fairer test would give each model the same total compute budget rather than the same number of epochs. Five model sizes for a three-parameter fit is tight, and more intermediate sizes would make the curve more trustworthy.

I ran into three non-obvious bugs. First, the training loop was overwriting `MuAdamW`'s per-group LR multipliers every step, invisible at Tiny but breaking wider models. Second, I initially used $1/d_{\text{head}}$ literally for μP attention scale instead of $\sqrt{\texttt{BASE_HEAD_DIM}}/d_{\text{head}}$, making attention too cold. Third, using the text string `"<svg"` as a generation prompt produced 0% render rate because the tokenizer encoded it differently than during training. I had to inject the correct token ID directly. All three bugs produced outputs that looked reasonable at first glance.

5.5 Future Work

There are a few things I would try with more time. The most obvious one is getting more multi-element SVGs - 12% is better than 0% but still not a lot, and pushing that above 30% would probably help prefix completion a lot. I would also train longer. Even 4 epochs is not that much, and the val loss was still dropping at the end. For μP, the right experiment would be to give each model the same total compute rather than the same number of epochs, since μP's lower effective LR needs more steps to show its benefit. A longer context window (2048 tokens instead of 1024) would probably help render rates too, since most of the unconditional failures are just the model running out of tokens before it can close its tags. Finally, the `umuthopeyildirim/svgen-500k` dataset has text descriptions paired with SVGs, which would let you try text-conditioned generation ("draw a star" $\rightarrow$ SVG output).

6 Conclusion

I trained five decoder-only transformers (roughly 800K to 85M non-embedding parameters) on 130M tokens of cleaned SVG code, fit scaling laws, compared SP against μP, and evaluated sample generation from an XL model retrained on a more diverse corpus.

The SP scaling law $L = 15.79 \cdot N^{-0.126} + 1.50$ fits well ($R^2 = 0.984$) across a 107$\times$ parameter range, with $\alpha = 0.126$ steeper than Kaplan et al.'s natural-language exponent. μP passed correctness checks but did not improve over SP in either model family, which I traced to its effective LR reduction under a tight step budget. For generation, rebalancing the corpus to include 12% multi-element SVGs was the single biggest change: the diversified model (perplexity 6.47) produces more interesting and varied outputs, though at a lower unconditional render rate due to context-window truncation. Overall, training data composition mattered more than model capacity for generation quality.

References

[1] Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models.* arXiv:2001.08361.

[2] Hoffmann, J. et al. (2022). *Training Compute-Optimal Large Language Models* (Chinchilla). arXiv:2203.15556.

[3] Yang, G. et al. (2022). *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer* (μP). arXiv:2203.09789.

[4] Rodriguez, J. A. et al. (2023). *StarVector: Generating Scalable Vector Graphics Code from Images and Text.* arXiv:2312.11556.

A Repository Structure

Path	Description
`scripts/01-04`	Data download, clean, tokenize, prepare
`scripts/01b_download_balanced.py`	Balanced corpus download
`scripts/05_train_model.py`	Train one SP model for 1 epoch
`scripts/06_lr_sweep.py`	SP LR sweep on Tiny
`scripts/06b_lr_sweep_width_only.py`	SP LR sweep on w_xs (width-only)
`scripts/07_train_mup.py`	Train one μP model for 1 epoch
`scripts/08_lr_sweep_mup.py`	μP LR sweep on Tiny
`scripts/08b_lr_sweep_mup_width_only.py`	μP LR sweep on w_xs (width-only)
`scripts/09_coord_check_mup.py`	μP coordinate check
`scripts/10-12`	Generate, evaluate, plot samples
`scripts/14_width_only_scaling.py`	Width-only family training and plotting
`src/model.py`	SP transformer with WIDTH_ONLY_CONFIGS
`src/model_mup.py`	μP transformer variant
`src/training_utils.py`	Training loop, LR schedule, checkpointing
`src/scaling_law.py`	Power-law fitting and extrapolation
`notebooks/01-04.ipynb`	Phase-by-phase walkthroughs
`Decision Log.md`	Full log of all design decisions

B Additional Generated Samples

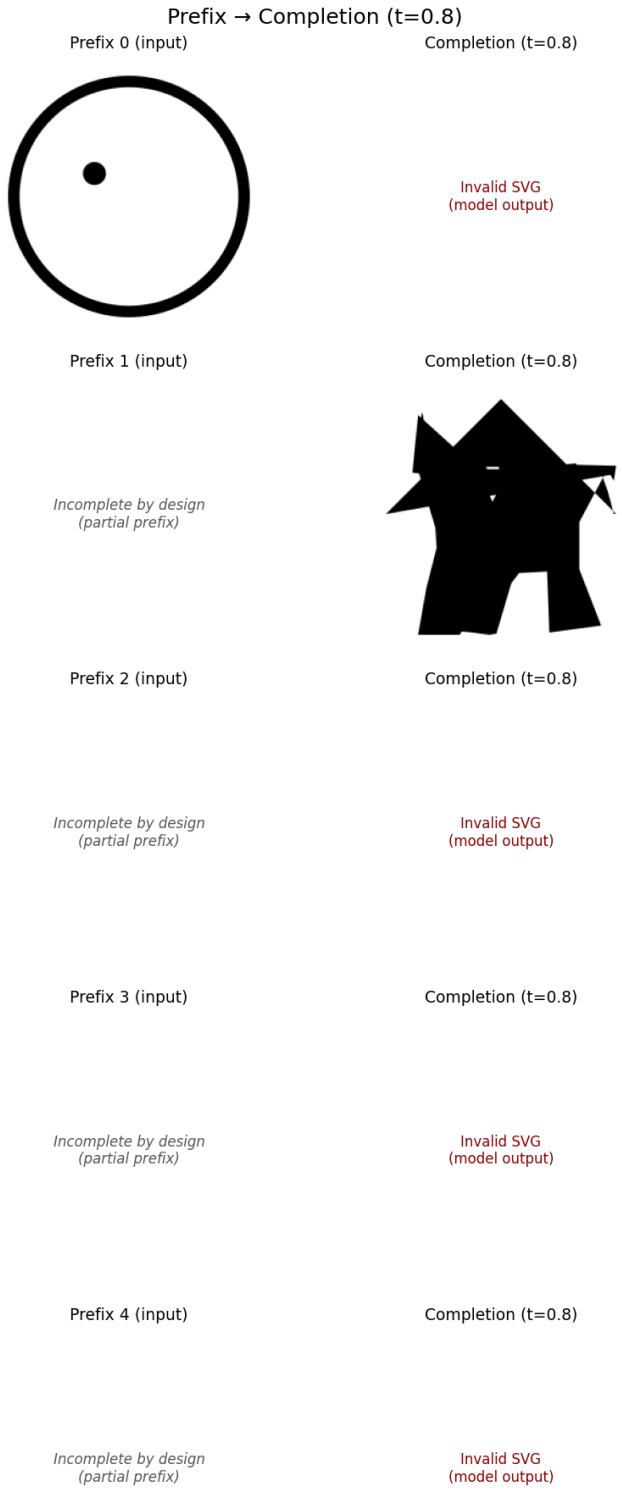


Figure 7: Original model (1-epoch, font-heavy corpus). Only Prefix 1 (open path) renders. All multi-element prefixes fail because training data contains only single-path SVGs.

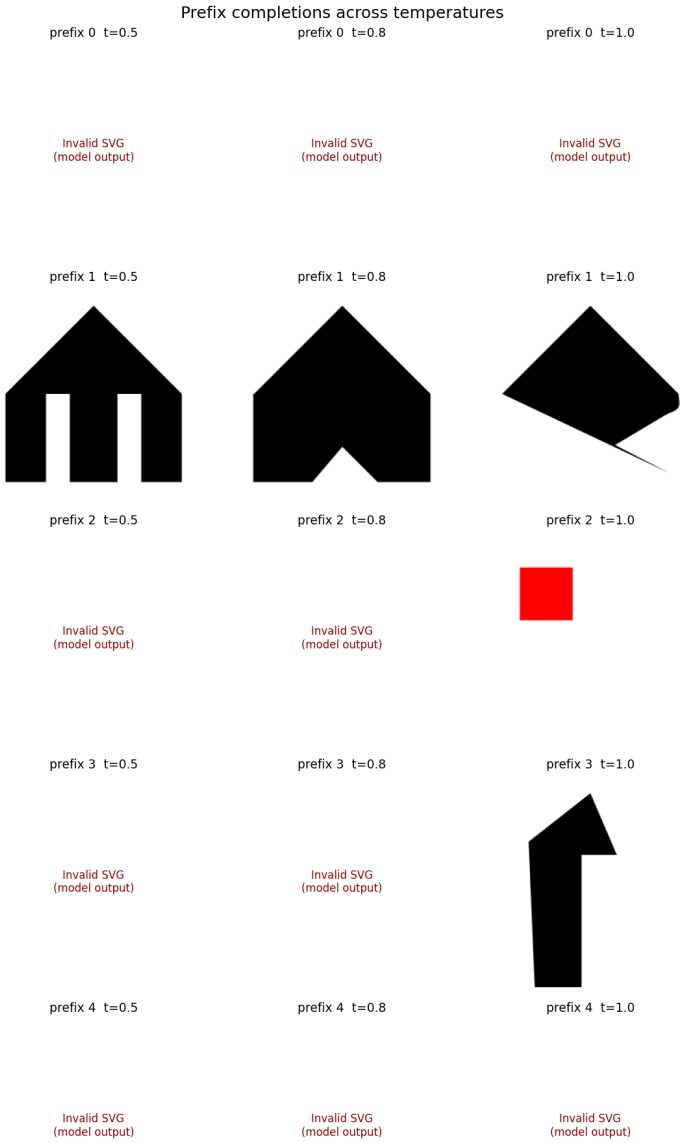


Figure 8: Diversified model: prefix completions across temperatures. Lower temperatures produce cleaner completions; $t = 1.0$ produces more varied but occasionally invalid output.

C Extended Original XL Samples (4 Epochs, Font-Heavy Corpus)

These samples are from the extended original XL, trained for 4 epochs on the font-heavy corpus (93% single-path glyphs). Comparing these to the 1-epoch originals (Figures 6(a) and (b)) shows that more training improved font-glyph quality but made multi-element prefix completion worse.

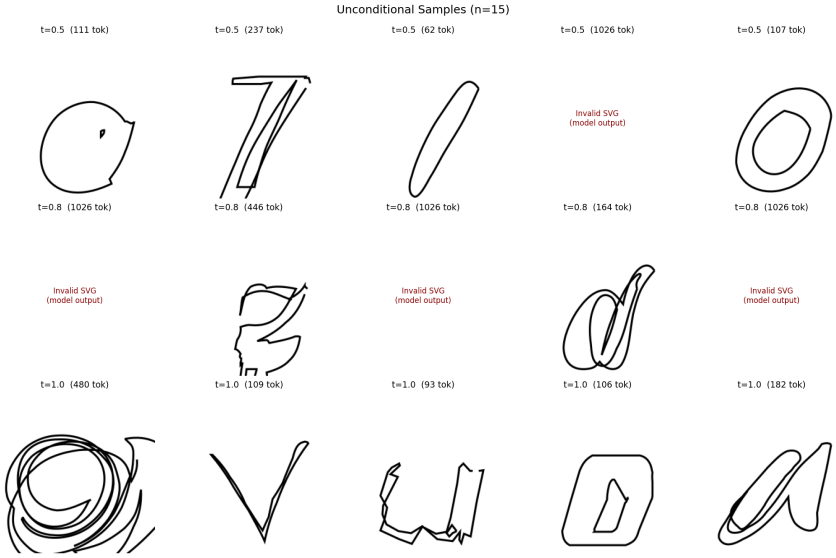


Figure 9: Unconditional samples from the extended original XL (4 epochs, font-heavy corpus). Render rate: 73.3% (11/15). Compared to the 1-epoch originals (Figure 6a), the outputs are noticeably cleaner: at $t = 0.5$ you can see a G-like shape, a 7, an I, and an O; at $t = 1.0$ there is a clear V, W, and D. The model got better at what it trained on (single-path font glyphs) but this improvement did not transfer to multi-element SVGs.

Figure 10: Prefix completions from the extended original XL (4 epochs, font-heavy corpus) at $t = 0.8$. All five prefixes fail. Compare to the 1-epoch original (Figure 7) where Prefix 1 at least rendered - more training on font-heavy data made prefix completion worse, not better.

Figure 11: Prefix completions from the extended original XL across temperatures. Render rate: 6.7% (1/15). Only Prefix 1 at $t = 1.0$ renders, producing a messy dark blob. All other prefixes at all temperatures fail. This confirmed that the bottleneck was data composition rather than training duration, and motivated building the diversified corpus.